

**«Санкт-Петербургский государственный электротехнический университет
«ЛЭТИ» им. В.И.Ульянова (Ленина)»
(СПбГЭТУ «ЛЭТИ»)**

**ОТЧЁТ
ПО УЧЕБНОЙ ПРАКТИКЕ**

**Тема: РАЗРАБОТКА ВИЗУАЛИЗАТОРА АЛГОРИТМА ДЕЙКСТРЫ
НА ЯЗЫКЕ JAVA**

Студент гр. 4344

М. Н. Сариев

Студент гр. 4345

Т.В. Мальцев

Студент гр. 4388

Г. Ю. Митин

Руководитель

М.А. Фирсов

Санкт-Петербург

2026

ЗАДАНИЕ НА УЧЕБНУЮ ПРАКТИКУ

Студент: М.Н. Сариев Группа: 4344

Студент: Т.В. Мальцев Группа: 4345

Студент: Г.Ю. Митин Группа: 4388

Тема практики: Разработка визуализатора алгоритма Дейкстры на языке Java.

Задание на практику:

Командная итеративная разработка визуализатора алгоритма(ов) на языке Java с графическим интерфейсом.

Алгоритм: алгоритм Дейкстры.

Сроки прохождения практики: 06.07.2026 – 18.07.2026

Дата сдачи отчета: 18.07.2026

Дата защиты отчета: 20.07.2026

Студент	М. Н. Сариев
Студент	Т. В. Мальцев
Студент	Г. Ю. Митин
Руководитель	М.А. Фирсов

АННОТАЦИЯ

Отчёт посвящён разработке визуализатора алгоритма Дейкстры на языке Java в рамках командной учебной практики. Приложение позволяет интерактивно строить ориентированный взвешенный граф, запускать на нём алгоритм Дейкстры в пошаговом либо мгновенном режиме и просматривать кратчайший путь между произвольными двумя вершинами.

Работа выполнялась бригадой из трёх студентов с распределением ролей: модель графа и алгоритм (бэкенд), графический интерфейс на Swing (фронтенд), тестирование. Разработка велась итеративно — прототип, версия 1, версия 2 — с согласованием и последующим уточнением спецификации по замечаниям руководителя.

В отчёте описаны постановка задачи, спецификация и план разработки, архитектура и ключевые классы приложения (модель графа, реализация алгоритма Дейкстры на основе очереди с приоритетом, компонент отрисовки и подсветки), а также подход к тестированию корректности алгоритма и графического интерфейса.

SUMMARY

The report describes the development of a Dijkstra's algorithm visualizer in Java, created as part of a team-based educational practice. The application allows the user to interactively build a directed weighted graph, run Dijkstra's algorithm in either step-by-step or instant mode, and inspect the shortest path between any two chosen vertices.

The work was carried out by a team of three students with the following role split: graph model and algorithm (backend), Swing-based graphical interface (frontend), and testing. Development followed an iterative process — prototype, version 1, version 2 — with the specification refined after review by the supervisor.

The report covers the problem statement, specification and development plan, the application architecture and key classes (graph model, a priority-queue-

based implementation of Dijkstra's algorithm, the rendering and highlighting component), and the approach taken to testing both algorithm correctness and the graphical interface.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

1.2 Задачи практики

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Вводное задание:

2.2. Составление спецификации программы и плана разработки

2.2.1 Исходные требования к программе

2.2.1.1 Требования к пользовательскому интерфейсу.

2.2.1.2 Требования к визуализации

2.2.1.3 Требования к текстовому сопровождению.

2.2.1.4 Схема графического интерфейса

2.2.1.6 Формат выходных данных

2.2.2 План разработки и распределение ролей в бригаде

2.2.3 Уточнение требований после сдачи прототипа

2.2.4 Уточнение требований после сдачи 1-й версии

2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом

2.3.1. Структуры данных

2.3.2. Основные методы (заголовок четвертого уровня)

2.3.3. Тестирование (заголовок четвертого уровня)

2.3.3.1 План тестирования

2.3.3.2 Тестирование графического интерфейса.

2.3.3.3 Тестирование алгоритма.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

4 ОТЗЫВ РУКОВОДИТЕЛЯ

ЗАКЛЮЧЕНИЕ

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ (заголовок второго уровня)

ПРИЛОЖЕНИЕ А

ПРИЛОЖЕНИЕ Б

ВВЕДЕНИЕ

Предметная область практики — разработка программного обеспечения с графическим интерфейсом на языке Java, а также алгоритмы на графах: поиск кратчайших путей во взвешенном ориентированном графе.

Цель практики — освоить язык Java и технологию командной итеративной разработки приложений с графическим интерфейсом на примере создания визуализатора классического алгоритма.

Задачи практики: выполнить вводное задание (изучить основы языка Java); составить и согласовать спецификацию и план разработки; в составе бригады итеративно разработать визуализатор алгоритма Дейкстры в соответствии со спецификацией.

1 ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область

Кратчайшие пути в графе — одна из базовых задач на графах: требуется найти путь минимального суммарного веса от заданной вершины до всех остальных (либо до конкретной вершины) во взвешенном графе. Задача возникает в маршрутизации, сетевом планировании, логистике и построении навигационных систем. Алгоритм Дейкстры решает эту задачу для графов с неотрицательными весами рёбер: заводится массив расстояний, изначально равных бесконечности (для стартовой вершины — нулю); на каждом шаге среди ещё не обработанных вершин выбирается вершина с минимальным известным расстоянием, помечается обработанной, а расстояния до её соседей уточняются (релаксируются), если путь через эту вершину оказывается короче уже известного [1]. Алгоритм завершается, когда все достижимые вершины обработаны; вершины, до которых нет пути из стартовой, остаются с расстоянием «бесконечность». При использовании очереди с приоритетом для выбора очередной вершины сложность алгоритма составляет $O((V + E) \log V)$, где V — число вершин, E — число рёбер. Визуализация делает наглядными работу очереди приоритетов и процесс релаксации рёбер, что облегчает понимание алгоритма при изучении курсов по структурам данных и алгоритмам.

1.2 Задачи практики

В рамках практики поставлены следующие задачи:

- задача 1: вводное задание — изучить основы языка Java по онлайн-курсу и ответить на вопросы преподавателя;
- задача 2: составить спецификацию визуализатора алгоритма Дейкстры и план итеративной разработки, распределить роли внутри бригады, согласовать с руководителем;
- задача 3: в составе бригады из трёх студентов итеративно разработать приложение-визуализатор в соответствии со

спецификацией и планом, сдавая промежуточные версии (прототип, версия 1, версия 2)

2 РЕЗУЛЬТАТЫ РАБОТЫ

2.1. Вводное задание:

Для вводного задания выбран второй вариант — изучение онлайн-курса «Java. Базовый курс» на платформе Stepik [2]. Полностью пройдены разделы 1–3, а также разделы 4.1, 4.2 и 5.1: примитивные и ссылочные типы данных, операторы, управляющие конструкции, массивы, методы, основы ООП (классы, инкапсуляция, наследование, полиморфизм, абстрактные классы и интерфейсы), исключения, коллекции, работа с файлами.

Помимо теоретических степов, решён ряд практических задач платформы.

2.2. Составление спецификации программы и плана разработки

Исходная спецификация и план разработки составлены бригадой и согласованы с руководителем; по замечаниям руководителя оба документа впоследствии дорабатывались. Ниже приведены требования и план с распределением ролей. Соответствующие документы также представлены в репозитории бригады [3].

2.2.1 Исходные требования к программе

2.2.1.1 Требования к пользовательскому интерфейсу.

- Создание графа, начиная с 1 вершины - Пользователь выбирает функцию создания графа вручную на панели управления, размещает вершину на доске, где отображается граф, дает вершине название.
- Добавление новых вершин в граф - Пользователь выбирает функцию добавления вершины на панели инструментов, далее см. пункт выше.
- Связывание вершин рёбрами заданной положительной длины - Пользователь выбирает функцию создания ребра на панели инструментов, в графе выбирает начальную вершину и конечную, задает длину дуги.

- Изменение длины существующих рёбер - Функция выбирается на панели инструментов, ребро выбирается на графе, редактируется длина.
- Удаление вершины со всеми инцидентными ей рёбрами - Функция выбирается на панели инструментов, далее выбирается вершина на графе.
- Удаление отдельного ребра - проводится аналогично пункту выше.
- Обработка некорректного ввода: отрицательная или нечисловая длина ребра, попытка создать петлю или мультиребро.
- Запуск алгоритма от выбранной вершины с расчётом кратчайших расстояний до всех остальных вершин графа - Пользователь выбирает функцию запуска алгоритма на панели управления, выбирает вершину на графе, затем выбирает режим работы программы.
- Просмотр кратчайшего пути между исходной и любой другой выбранной вершиной после завершения работы алгоритма - выбирается кнопка на панели инструментов, выбирается вершина на графе.
- Выбор режима работы алгоритма: пошаговый (вручную) и мгновенный (сразу весь результат) - Пользователь выбирает режим, нажимая соответствующую кнопку на панели управления.

2.2.1.2 Требования к визуализации

- Создание графа, начиная с 1 Отображение графа в виде именованных вершин, соединённых рёбрами с подписанными весами
- Во время пошаговой работы алгоритма: подсветка текущей рассматриваемой вершины и смежных с ней вершин, на которых происходит пересчёт

- Отображение текущих (промежуточных) значений расстояний рядом с вершинами по ходу работы алгоритма
- Подсветка уже посещённых вершин, для которых кратчайшее расстояние окончательно определено
- Отображение дерева кратчайших путей для уже посещенных вершин путем подсветки ребер, составляющих это дерево, на графе
- Явный сигнал о завершении работы алгоритма
- Подсветка рёбер, образующих кратчайший путь, после завершения работы алгоритма

2.2.1.3 Требования к текстовому сопровождению.

- Оповещение о старте алгоритма с указанием стартовой вершина
- В ходе пошаговой работы алгоритма на каждом шаге, начиная:
- Вывод в панель сообщений текущей очереди с приоритетом вершин, которые возможно посетить на следующем шаге
- Вывод в панель сообщений длины пути до вершины, которая выбирается, как ближайшая из непосещенных
- Оповещение о конце работы алгоритма с указанием, удалось ли обойти весь граф, или есть вершины за пределами рассмотренной области связности. Выводит найденные пути.
- Текстовый вывод найденного пути и его суммарной длины при выборе пользователем данной функции после работы алгоритма

2.2.1.4 Схема графического интерфейса

Схема интерфейса представлена на рисунке ниже. (см. рис. 1)

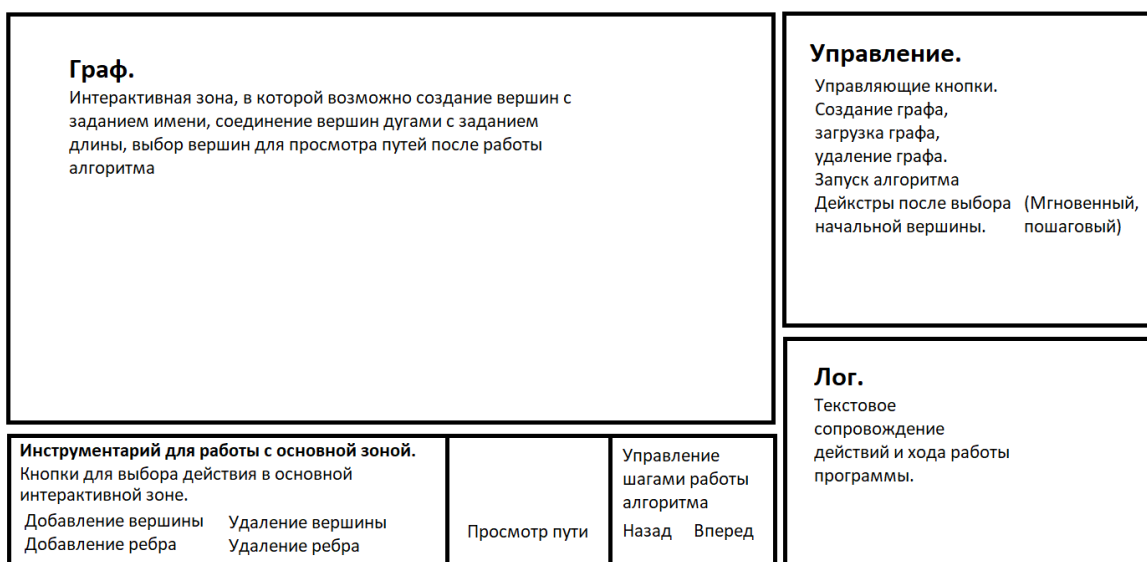


Рисунок 1 – Схема интерфейса приложения.

2.2.1.5 Формат входных данных

Вершины и рёбра задаются пользователем интерактивно через графический интерфейс. Вершины создаются по кнопке и именуются. Рёбра создаются между двумя существующими вершинами с указанием положительной вещественной длины (десятичный разделитель — точка).

Альтернативно, файл json-формата, содержащий список смежности, задающий граф.

Пример:

```
{
  "vertices": [
    { "id": "A", "name": "Прямо и налево" },
    { "id": "B", "name": "Налево" },
    { "id": "C", "name": "Направо" },
    { "id": "D", "name": "Направо дважды" },
    { "id": "E", "name": "Вперед" }
  ],
  "adjacency": {
    "A": [
      { "to": "B", "weight": 5 },
```

```

    { "to": "C", "weight": 3 }
  ],
  "B": [
    { "to": "D", "weight": 7 }
  ],
  "C": [
    { "to": "D", "weight": 2 },
    { "to": "E", "weight": 4 }
  ],
  "D": [
    { "to": "E", "weight": 6 }
  ],
  "E": []
}
}

```

2.2.1.6 Формат выходных данных

Список кратчайших расстояний от исходной вершины до всех остальных, например:

Вершина A → Вершина B — 12.2, Вершина A → Вершина C — 33.4, ..., Вершина A → Вершина P — 76.67

Кратчайший путь между двумя выбранными вершинами в графическом виде (подсветка) и текстом, например:

*Вершина A → Вершина D (14) → Вершина E (8.7) → Вершина C (1).
Длина пути: 23.7*

2.2.2 План разработки и распределение ролей в бригаде

Распределение ролей при разработке представлено в таблице 1.

Таблица 1 – Распределение ролей

Участник	Роль	Зона ответственности
Сариев Максим	Бэкенд	Модель графа (Vertex, Edge, Graph), реализация алгоритма

Участник	Роль	Зона ответственности
		Дейкстры
Тимур Мальцев	Фронтенд + Бэкэнд	Графический интерфейс: отрисовка графа, подсветка вершин и ребер
Митин Георгий	Тестирование + Фронтэнд	План тестирования, кнопки интерфейса, панель текстовых пояснений

График сдач версий представлен в таблице 2.

Таблица 2 - График сдач

Дата	Этап	Что сдаём
10 июля	Спецификация	Согласование спецификации и плана, репозиторий создан и отправлен преподавателю
13 июля (пн)	Прототип	Срок сдачи прототипа, скелет приложения
15 июля (ср)	Версия 1	Алгоритм Дейкстры реализован и корректно работает пошагово, выводит текстовые данные
17 июля (пт)	Версия 2	Полная визуализация: подсветка, путь между вершинами, валидация, режимы шаг/мгновенно

2.2.3 Уточнение требований после сдачи прототипа

- Размещаемые пользователем вершины графа на холсте не могут накладываться друг на друга. При попытке размещения всплывает предупреждение, и вершина не создается

- При запуске алгоритма после выбора соответствующей функции пользователю нужно нажать на вершину, которую он хотел бы сделать начальной для алгоритма, на холсте мышью
- Функция очистки графа. С помощью нажатия кнопки на панели управления очистить выделение вершин и ребер, которое произошло в результате выполнения любого числа шагов алгоритма, запущенного ранее
- Изменение формата json-файла. Только блок со списком смежности вершин. Пример:

```
{
  "A": [
    { "to": "B", "weight": 5 },
    { "to": "C", "weight": 3 }
  ],
  "B": [
    { "to": "D", "weight": 7 }
  ],
  "C": [
    { "to": "D", "weight": 2 },
    { "to": "E", "weight": 4 }
  ],
  "D": [
    { "to": "E", "weight": 6 }
  ],
  "E": []
}
```

2.2.4 Уточнение требований после сдачи 1-й версии

- Поле текстовых сообщений сделать редактируемым.
- Редактирование панели текстовых сообщений

- Перемещение вершин на графе с сохранением их свойств.
- Корректное отображение длин противоположно направленных ребер

2.3. Командная итеративная разработка приложения в соответствии со спецификацией и планом

Разработка велась в целом в соответствии с согласованным планом: прототип содержал только модель графа и статичную отрисовку, версия 1 — пошаговый алгоритм без визуализации, версия 2 — полную визуализацию, валидацию ввода и оба режима работы.

2.3.1. Структуры данных

Vertex — вершина графа: имя и координаты на канвасе; равенство определено по имени. Edge — направленное ребро: ссылки на начальную и конечную вершины и вес. Graph хранит списки вершин и рёбер и проверяет ограничения спецификации при добавлении (запрет петель, мультирёбер, неположительного веса, слишком близкого расположения вершин), предоставляет выборку рёбер, исходящих из вершины.

DijkstraAlgorithm хранит текущие расстояния, множество посещённых вершин, дерево предшественников и очередь с приоритетом VertexDistance («вершина — расстояние»); отдельно ведётся история снимков состояния по шагам (StepState), позволяющая перемещаться по уже пройденным шагам вперёд и назад без повторных вычислений. PathResult хранит результат восстановления кратчайшего пути — последовательность вершин и его суммарную длину.

2.3.2. Основные методы

DijkstraAlgorithm.step — шаг вперёд: если пользователь уже возвращался назад по истории, переход к ранее вычисленному снимку без пересчёта; иначе выполняется реальное вычисление — извлечение из очереди с приоритетом ближайшей ещё не посещённой вершины, релаксация исходящих рёбер. DijkstraAlgorithm.stepBack — перемещение указателя на

снимок состояния на шаг назад. `DijkstraAlgorithm.getPath` — восстановление кратчайшего пути обратным проходом по дереву предшественников.

`GraphPanel.paintComponent` — отрисовка графа со стрелками, подсветкой вершин и рёбер по состоянию алгоритма (посещена / в очереди / текущая / часть кратчайшего пути) и подписями расстояний. Обработчики мыши `GraphPanel` создают, перемещают и удаляют элементы графа. `JsonLoader.load` — разбор JSON-файла графа с использованием библиотеки `Jackson` и автоматическая генерация непересекающихся координат вершин. `MainFrame` связывает кнопки панели инструментов, диалог выбора режима и журнал с моделью и алгоритмом.

2.3.3. Тестирование

Тестирование выполнялось на трёх уровнях, описанных ниже.

2.3.3.1 План тестирования

Для модели графа проверялось: сохранение вершины/ребра при добавлении, отклонение дублирующегося имени вершины, петли, неположительного веса и мультиребра; при удалении — что пропадает только удаляемый элемент (для вершины — вместе со всеми инцидентными рёбрами), а удаление несуществующего элемента не приводит к ошибке. Для окна проверялось открытие при запуске, завершение программы при закрытии окна, соответствие размера ожидаемому, отображение панели инструментов и области отрисовки. Для алгоритма Дейкстры проверялось, что после каждого шага корректно обновляются расстояния, очередь и выбор следующей вершины — на графах, перечисленных в таблице 3 (от стартовой вершины A).

Таблица 3 – Тестовые случаи

Текстовый граф	Ожидаемый результат
Одна вершина A, без рёбер	A = 0
Цепь: A→B (5)	A = 0, B = 5

Два пути: $A \rightarrow B$ (5), $A \rightarrow C$ (10), $B \rightarrow C$ (3)	$A = 0, B = 5, C = 8$
Недостижимая вершина: $A \rightarrow B$ (5), C без входящих рёбер	$A = 0, B = 5, C = \infty$
Направленный цикл: $A \rightarrow B$ (5), $B \rightarrow C$ (5), $C \rightarrow A$ (5)	$A = 0, B = 5, C = 10$

Отдельно проверялось, что направленный цикл не приводит к заикливанию самого алгоритма, что после завершения работы появляется сообщение о завершении и что алгоритм может быть запущен повторно от другой стартовой вершины. Для визуализации проверялась корректность обработки исключительных ситуаций при вводе некорректных данных и корректность подсветки текущей вершины и её соседей, отображения промежуточных расстояний, дерева кратчайших путей и итогового кратчайшего пути.

2.3.3.2 Тестирование графического интерфейса.

Интерфейс проверен вручную по сценариям плана: создание и удаление вершин и рёбер, перемещение вершины, обработка некорректного ввода, запуск алгоритма в пошаговом и мгновенном режимах, навигация по истории шагов вперёд и назад, просмотр пути между произвольными вершинами, загрузка графа из JSON-файла. Все перечисленные сценарии проверены на нескольких графах разного размера и отработали корректно. Результаты приведены в приложении Б.

2.3.3.3 Тестирование алгоритма.

Модель и алгоритм проверены автономными тестами без графического интерфейса. На графе-«ромбе» из четырёх вершин (рёбра $A \rightarrow B$ (4), $A \rightarrow C$ (1), $C \rightarrow B$ (2), $B \rightarrow D$ (2)) получены расстояния $\{A=0, B=3, C=1, D=5\}$ и путь

$A \rightarrow C \rightarrow B \rightarrow D$, то есть алгоритм предпочёл более длинный по числу рёбер, но более короткий по суммарному весу маршрут прямому ребру $A \rightarrow B$. На графе из шести вершин с несколькими альтернативными путями результат $\{A=0, B=2, C=5, D=4, F=5, E=6\}$ полностью совпал с расчётом вручную. Дополнительно проверено, что граф действительно направленный (путь из «конечной» вершины в «стартовую» корректно определяется как недостижимый при отсутствии обратных рёбер) и что навигация по истории шагов назад и вперёд не искажает итоговый результат по сравнению с прямым выполнением без промежуточных откатов. Все тесты пройдены.

3 ИСПОЛЬЗУЕМЫЕ ТЕХНОЛОГИИ

Java 21 (сборка Eclipse Temurin) — язык разработки. Swing (входит в JDK) — библиотека графического интерфейса: JFrame, JSplitPane, JPanel, отрисовка через Graphics2D, диалоги JOptionPane и JFileChooser. Стандартная коллекция java.util.PriorityQueue — очередь с приоритетом для алгоритма [4]. Сторонняя библиотека Jackson (com.fasterxml.jackson) — разбор JSON при загрузке графа из файла.

Совместная разработка велась с использованием системы контроля версий Git и общего репозитория бригады на GitHub. Среда разработки — Visual Studio Code с Extension Pack for Java; сборка и запуск на этапе разработки — средствами среды, напрямую через javac и java из командной строки, при помощи системы сборки Maven.

4 ОТЗЫВ РУКОВОДИТЕЛЯ

По результатам работы учебная практика оценена на **Хорошо**.

Отзыв руководителя с оценкой. (см. рис. 2)

Отзыв

о прохождении учебной практики

Сариев Максим Николаевич, студент группы 4344 второго курса бакалавриата Санкт-Петербургского электротехнического университета “ЛЭТИ” им В.И. Ульянова, проходил учебную практику по теме “Новые языки программирования” с 06.07.2026 по 18.07.2026 на кафедре МО ЭВМ. Практика включала освоение нового языка программирования и командную итеративную разработку на нём визуализатора алгоритма в соответствии со спецификацией и планом разработки.

Во время практики Сариев Максим Николаевич выполнил поставленные задачи, продемонстрировал владение языком программирования Java, умение планировать разработку и работать в команде. К работе студента во время практики возникли некоторые замечания.

По результатам работы на учебной практике Сариев Максим Николаевич заслуживает оценки **Хорошо**.

Руководитель практики

Ст. преподаватель кафедры МОЭВМ 18.07.2026  / Фирсов М.А.

Рисунок 2 – Отзыв руководителя.

ЗАКЛЮЧЕНИЕ

В ходе учебной практики решены все поставленные задачи. Выполнено вводное задание: изучен базовый курс Java на платформе Stepik. Составлена и, по замечаниям руководителя, дважды доработана спецификация визуализатора алгоритма Дейкстры с планом итеративной разработки и распределением ролей внутри бригады.

Бригадой из трёх студентов разработано приложение с графическим интерфейсом: модель графа с валидацией ввода, пошаговая реализация алгоритма Дейкстры на основе очереди с приоритетом с возможностью навигации по истории шагов вперёд и назад, визуализация работы алгоритма с подсветкой вершин и рёбер и текстовыми пояснениями, восстановление и отображение кратчайшего пути между произвольными вершинами, загрузка графа из JSON-файла.

Корректность работы алгоритма подтверждена тестами на графах с заранее известным правильным ответом, включая проверку направленности графа и согласованности навигации по истории шагов. Практика позволила освоить язык Java, библиотеку Swing и технологию командной итеративной разработки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Алгоритмы: построение и анализ / Т. Кормен и др. — 3-е изд. — М.: Вильямс, 2013. — 1328 с.
2. Java. Базовый курс [Электронный ресурс]. URL: <https://stepik.org/course/Java-Базовый-курс-187/syllabus> (дата обращения: 07.07.2026).
3. Репозиторий GitHub бригады [Электронный ресурс]. URL: https://github.com/timmaltsev/Prac_16_Dijkstra_java .
4. Java Platform, Standard Edition 21 API Specification [Электронный ресурс]. URL: <https://docs.oracle.com/en/java/javase/21/docs/api/> (дата обращения: 07.07.2026).

ПРИЛОЖЕНИЕ А

Ниже приведён листинг ключевого класса приложения — `algorithm/DijkstraAlgorithm.java`, реализующего пошаговый алгоритм Дейкстры с очередью с приоритетом и историей шагов. Полный исходный код всех классов приложения находится в репозитории проекта.

```
package algorithm;

import model.Edge;
import model.Graph;
import model.Vertex;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.PriorityQueue;
import java.util.Set;

/**
 * Пошаговая реализация алгоритма Дейкстры.
 *
 * Изменения в этой версии:
 * 1. В историю снимков (StepState) добавлен predecessors –
раньше туда попадали
 * только distances/visited/queue, а predecessors читался
из "живого" поля.
 * Из-за этого при stepBack() расстояния откатывались
корректно, а подсветка
 * дерева кратчайших путей (tree/candidate рёбра в
GraphPanel) – нет, потому что
 * getPredecessors() отдавал самое АКТУАЛЬНОЕ состояние,
а не то, что на снимке.
 * 2. finishAlgorithm() теперь идемпотентен (флаг
summaryLogged) – раньше при
 * повторном вызове (а он вызывался дважды в INSTANT-
режиме: один раз изнутри
 * runToCompletion(), второй раз явно из MainFrame)
итоговая сводка расстояний
 * дублировалась тестом.
 */
public class DijkstraAlgorithm {
    private final List<String> log = new ArrayList<>();
    private int lastLogIndex = 0;
    private int stepNumber = 0;
```

```

        private final Graph graph;
        private final Vertex source;

        // "рабочее" состояние – то, чем реально считает
        алгоритм, движется только вперёд
        private final Map<Vertex, Double> distances = new
        HashMap<>();
        private final Map<Vertex, Vertex> predecessors = new
        HashMap<>();
        private final Set<Vertex> visited = new HashSet<>();
        private boolean finished = false;
        private boolean summaryLogged = false;

        // приоритетная очередь для быстрого выбора следующей
        вершины (ленивое удаление устаревших записей)
        private final PriorityQueue<VertexDistance> queue =
            new
        PriorityQueue<>(Comparator.comparingDouble(VertexDistance::getDi
        stance));

        // история снимков состояния – по ней листает
        stepBack()/step(), не влияет на вычисления
        private final List<StepState> history = new
        ArrayList<>();
        private int viewIndex;

        private static final class StepState {
            final Vertex currentVertex;
            final Map<Vertex, Double> distances;
            final Set<Vertex> visited;
            final Map<Vertex, Vertex> predecessors;
            final ArrayList<VertexDistance> queue;

            StepState(Vertex currentVertex, Map<Vertex, Double>
            distances, Set<Vertex> visited,
                    Map<Vertex, Vertex> predecessors,
            ArrayList<VertexDistance> queue) {
                this.currentVertex = currentVertex;
                this.distances = distances;
                this.visited = visited;
                this.predecessors = predecessors;
                this.queue = queue;
            }
        }

        public DijkstraAlgorithm(Graph graph, Vertex source) {
            this.graph = graph;
            this.source = source;

            for (Vertex vertex : graph.getVertices()) {
                distances.put(vertex, Double.POSITIVE_INFINITY);
            }
        }
    }

```

```

    }
    distances.put(source, 0.0);
    queue.add(new VertexDistance(source, 0.0));

    history.add(snapshot(null));
    viewIndex = 0;
}

private StepState snapshot(Vertex currentVertex) {
    return new StepState(
        currentVertex,
        new HashMap<>(distances),
        new HashSet<>(visited),
        new HashMap<>(predecessors),
        new ArrayList<>(queue)
    );
}

/**
 * Шаг вперёд. Если пользователь до этого нажимал
"назад" и мы находимся
 * внутри уже посчитанной истории – просто листаем её
дальше, без пересчёта.
 * Если мы на границе истории – реально считаем новый
шаг алгоритма.
 */
public boolean step() {
    if (viewIndex < history.size() - 1) {
        viewIndex++;
        addLog("\u2192 Шаг вперёд (повтор из истории): "
+ describeState(history.get(viewIndex)));
        return true;
    }
    boolean computed = computeStep();
    if (computed) {
        viewIndex = history.size() - 1;
    }
    return computed;
}

/**
 * Шаг назад – листает историю снимков, вычисления не
трогает.
 */
public boolean stepBack() {
    if (viewIndex <= 0) {
        return false;
    }
    viewIndex--;
    addLog("\u2190 Шаг назад: " +
describeState(history.get(viewIndex)));
    return true;
}

```

```

    }

    private String describeState(StepState state) {
        return state.currentVertex == null
            ? "начальное состояние (до первого шага)"
            : "вершина " +
state.currentVertex.getName();
    }

    public boolean canStepForward() {
        return viewIndex < history.size() - 1 || !finished;
    }

    public boolean canStepBack() {
        return viewIndex > 0;
    }

    private boolean computeStep() {
        if (finished) {
            return false;
        }

        Vertex next = null;
        while (!queue.isEmpty()) {
            VertexDistance candidate = queue.poll();
            if (!visited.contains(candidate.getVertex())) {
                next = candidate.getVertex();
                break;
            }
        }

        if (next == null) {
            finished = true;
            addLog("Алгоритм завершён: оставшиеся вершины
недостижимы из " + source.getName());
            history.add(snapshot(null));
            return true;
        }

        stepNumber++;

        addLog("Шаг " + stepNumber);
        addLog("Рассматриваемая вершина: " +
next.getName());
        visited.add(next);
        addLog("Очередь: " + formatQueue());
        addLog("Путь: " + formatPath(next));

        for (Edge edge : graph.getOutgoingEdges(next)) {
            Vertex neighbor = edge.getTo();

```

```

        if (visited.contains(neighbor)) {
            continue;
        }

        addLog("Проверяем вершину " +
neighbor.getName());

        double newDistance = distances.get(next) +
edge.getWeight();

        if (newDistance < distances.get(neighbor)) {

            addLog("Расстояние обновлено: "
                + distances.get(neighbor)
                + " -> "
                + newDistance);

            distances.put(neighbor, newDistance);
            predecessors.put(neighbor, next);
            queue.add(new VertexDistance(neighbor,
newDistance));

        } else {

            addLog("Расстояние не изменилось.");

        }
    }

    addLog("Обновленная очередь: " + formatQueue());
    addLog("");

    if (visited.size() == graph.getVertices().size()) {
        finished = true;
    }

    history.add(snapshot(next));
    return true;
}

private String formatQueue() {

    List<Vertex> queueView = new ArrayList<>();

    for (Vertex vertex : graph.getVertices()) {
        if (!visited.contains(vertex)) {
            queueView.add(vertex);
        }
    }
}

```

```

        queueView.sort((v1, v2) ->
            Double.compare(distances.get(v1),
distances.get(v2)));

        StringBuilder builder = new StringBuilder();

        for (Vertex vertex : queueView) {

            if (builder.length() > 0) {
                builder.append(", ");
            }

            builder.append(vertex.getName())
                .append(" (");

            double distance = distances.get(vertex);

            if (Double.isInfinite(distance)) {
                builder.append("\u221e");
            } else {
                builder.append(distance);
            }

            builder.append(")");
        }

        return builder.toString();
    }

    private String formatPath(Vertex target) {

        List<Vertex> path = new ArrayList<>();

        Vertex current = target;

        while (current != null) {
            path.add(current);
            current = predecessors.get(current);
        }

        Collections.reverse(path);

        StringBuilder builder = new StringBuilder();

        for (Vertex vertex : path) {

            if (builder.length() > 0) {
                builder.append(" -> ");
            }

```

```

        }

        builder.append(vertex.getName());
    }

    return builder.toString();
}

/**
 * Выполняет все оставшиеся шаги разом – используется
для режима "Мгновенно".
 */
public void runToCompletion() {
    while (canStepForward()) {
        step();
    }
    finishAlgorithm();
}

public PathResult getPath(Vertex target) {
    if (!distances.containsKey(target) ||
distances.get(target) == Double.POSITIVE_INFINITY) {
        return new PathResult(Collections.emptyList(),
Double.POSITIVE_INFINITY);
    }

    List<Vertex> path = new ArrayList<>();
    Vertex step = target;
    while (step != null) {
        path.add(step);
        step = predecessors.get(step);
    }
    Collections.reverse(path);

    return new PathResult(path, distances.get(target));
}

// ==== геттеры состояния – читают из ИСТОРИИ
(viewIndex), чтобы stepBack()
//      реально откатывал ВСЮ картину разом, включая
дерево путей ====

public Vertex getCurrentVertex() {
    return history.get(viewIndex).currentVertex;
}

public Map<Vertex, Double> getDistances() {
    return history.get(viewIndex).distances;
}

```

```

public Set<Vertex> getVisited() {
    return history.get(viewIndex).visited;
}

public Map<Vertex, Vertex> getPredecessors() {
    return history.get(viewIndex).predecessors;
}

public boolean isInQueue(Vertex vertex) {
    for (VertexDistance point :
history.get(viewIndex).queue) {
        if (point.getVertex().equals(vertex))
            return true;
    }
    return false;
}

public boolean isFinished() {
    return finished;
}

public Vertex getSource() {
    return source;
}

private void addLog(String message) {
    log.add(message);
}

public List<String> consumeLog() {
    List<String> result = new ArrayList<>(
        log.subList(lastLogIndex, log.size())
    );
    lastLogIndex = log.size();
    return result;
}

/**
 * Печатает итоговую сводку текстом. Идемпотентен –
повторные вызовы
 * ничего не делают, чтобы сводка не задваивалась
(раньше вызывался
 * и изнутри runToCompletion(), и отдельно из
MainFrame).
 */
public void finishAlgorithm() {

    finished = true;

    if (summaryLogged) {
        return;
    }

```

```

    }
    summaryLogged = true;

    addLog("");
    addLog("Алгоритм завершен.");

    if (visited.size() == graph.getVertices().size()) {
        addLog("Все вершины графа были достигнуты.");
    } else {
        addLog("Граф полностью не достигим из стартовой
вершины.");
        addLog("Посещено вершин: "
            + visited.size()
            + " из "
            + graph.getVertices().size());
    }

    addLog("");
    addLog("Итоговые кратчайшие расстояния:");

    List<Vertex> vertices = new
ArrayList<>(graph.getVertices());
    vertices.sort((v1, v2) ->
v1.getName().compareTo(v2.getName()));

    for (Vertex vertex : vertices) {

        double distance = distances.get(vertex);

        if (Double.isInfinite(distance)) {
            addLog(vertex.getName() + " : недостижима");
        } else {
            addLog(vertex.getName() + " : " + distance);
        }
    }
}

```


ПРИЛОЖЕНИЕ Б

В приложении Б приведены результаты тестирования графического интерфейса.

Начало работы. Загружен граф. Ребра и вершины отображаются корректно. (см. рис. 3)

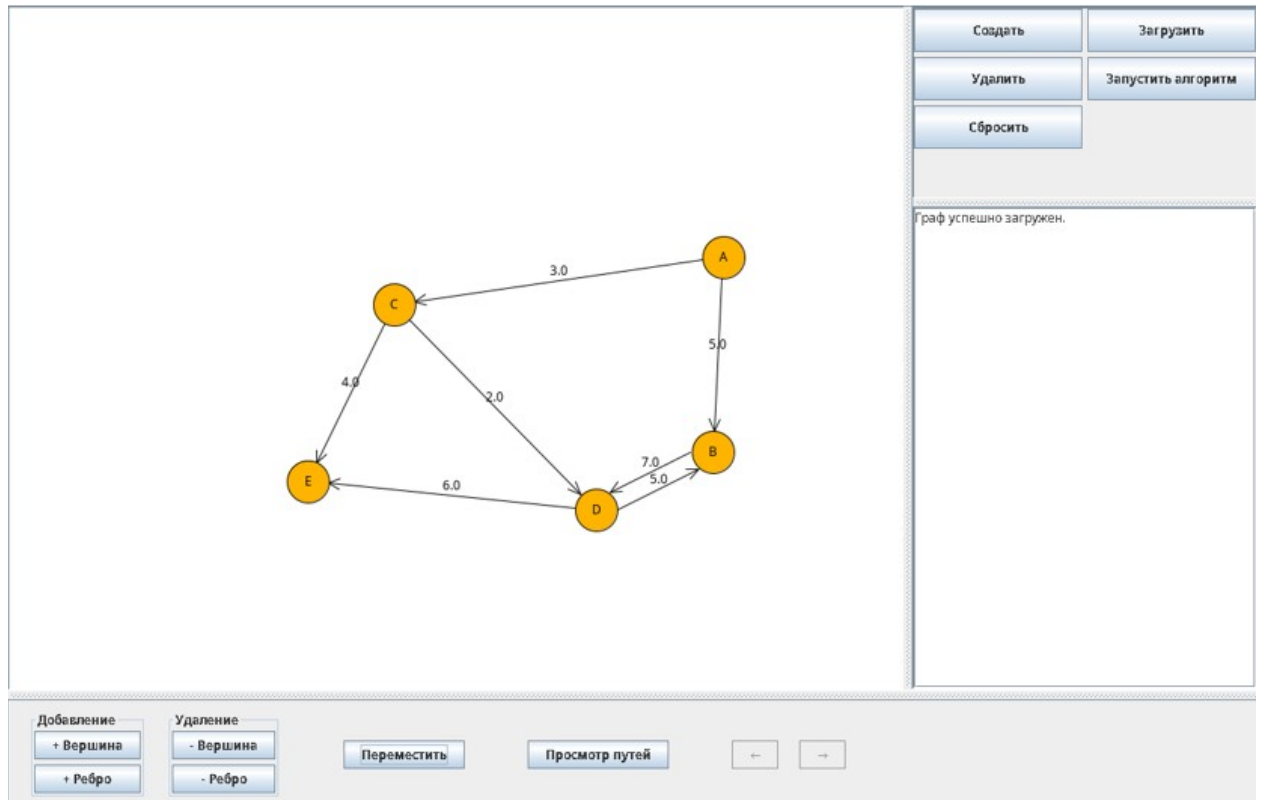


Рисунок 3 – Загруженный граф.

При удалении вершины D удаляются все инцидентные ей ребра.

Удаление ребер и вершин работает корректно. Вершины перемещаются. (см. рис. 4)

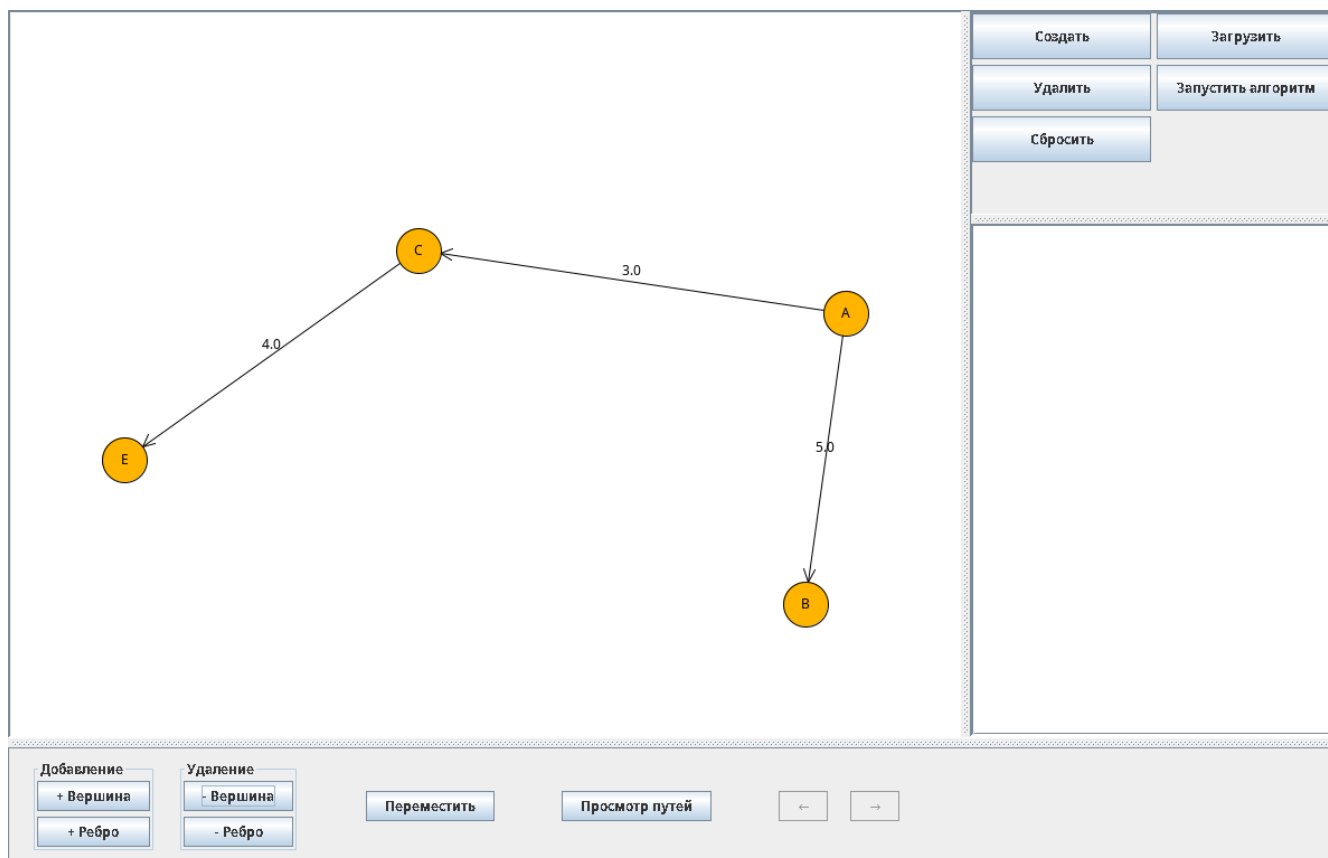


Рисунок 4 – удаление вершины.

Мгновенный режим работы алгоритма со стартовой вершины A и показ пути до вершин E. (см. рис. 5)

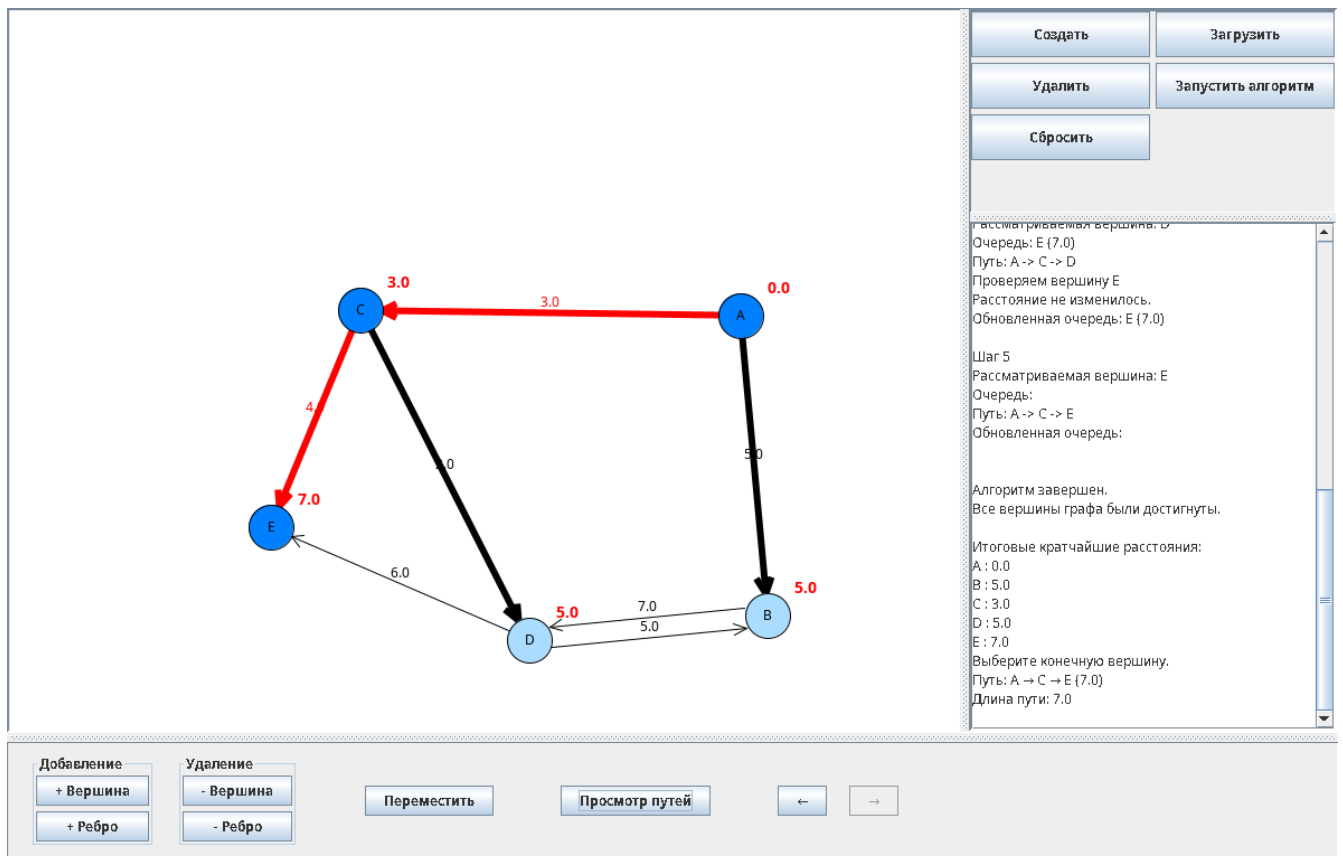
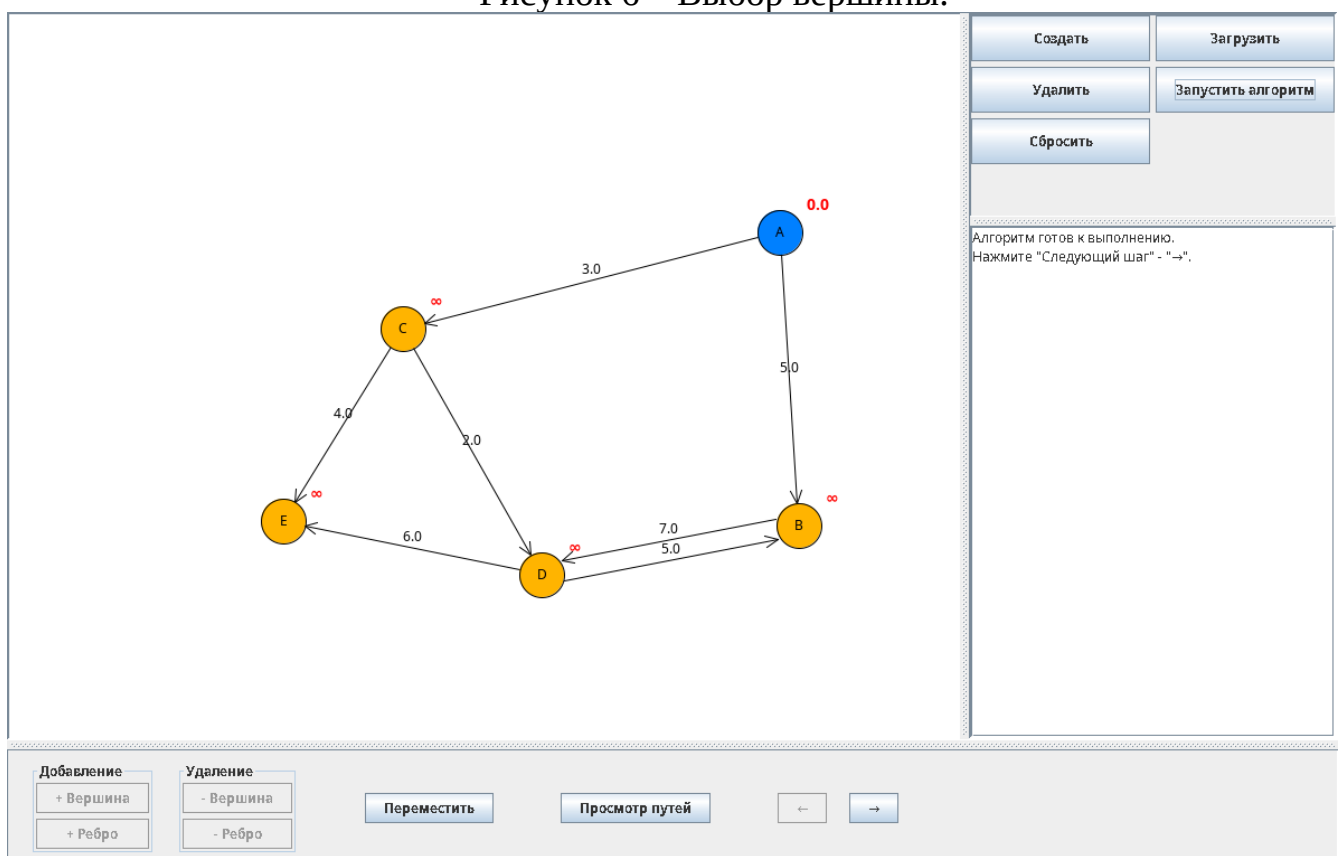


Рисунок 5 – мгновенный режим.

Пошаговый режим.

Выбрана начальная вершина A. (см. рис. 6)

Рисунок 6 – Выбор вершины.



Рассматриваются смежные вершины, обновляются расстояния. (см. рис. 7)

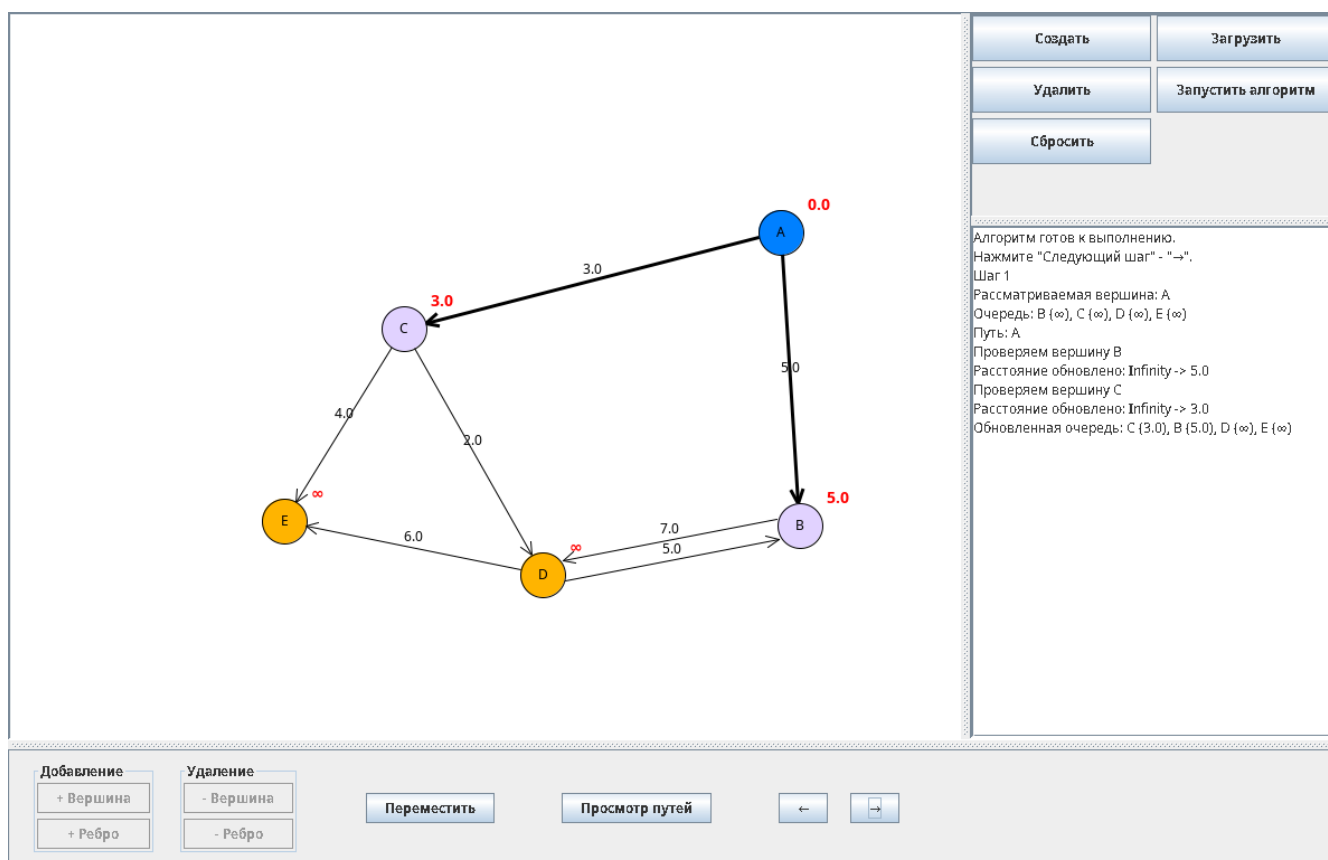


Рисунок 7 – первый шаг алгоритма.

Переход к первой смежной вершине. (см. рис. 8)

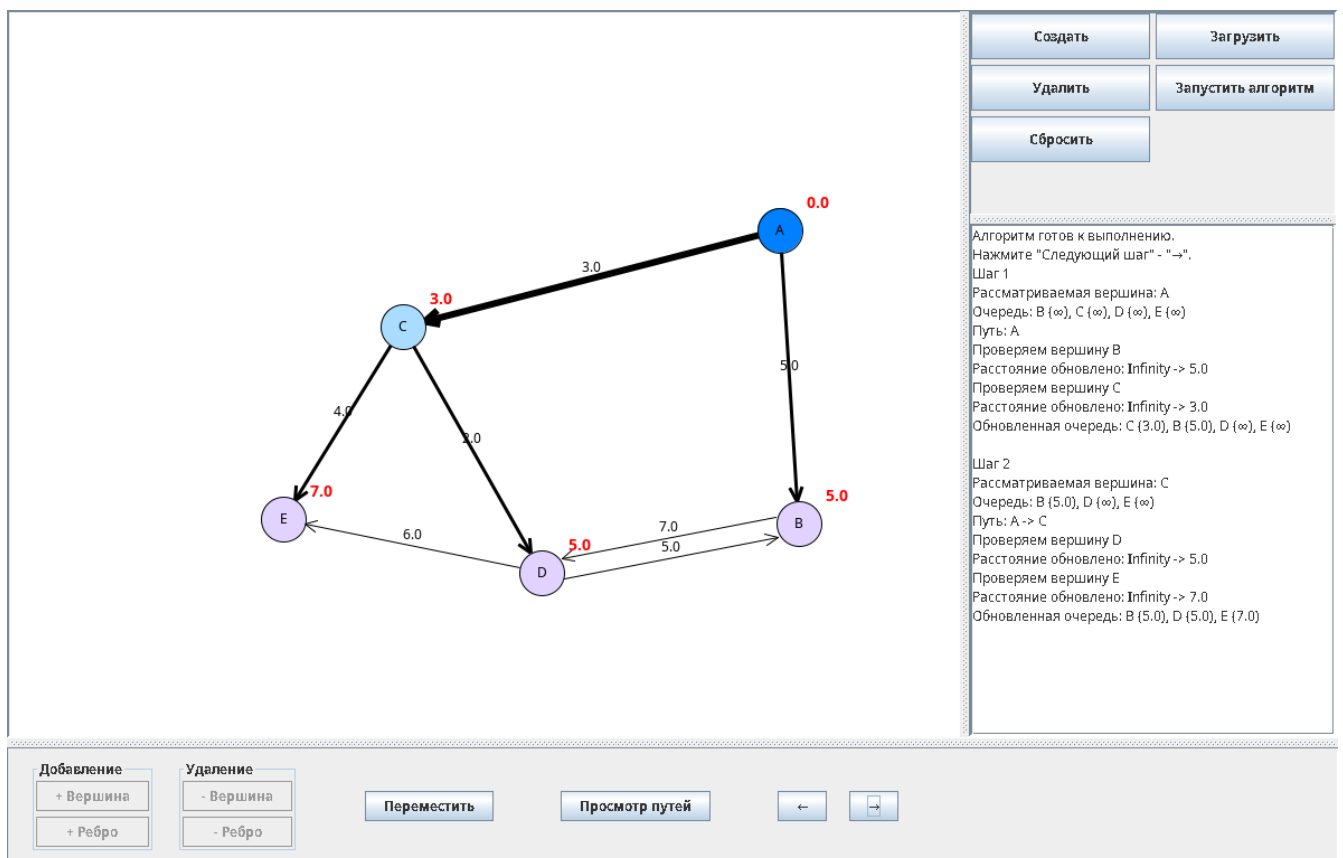


Рисунок 8 – Второй шаг.

Переход ко второй смежной вершине. Расстояние от В к D не обновилось, так как оно больше, уже имеющегося. (см. рис. 9)

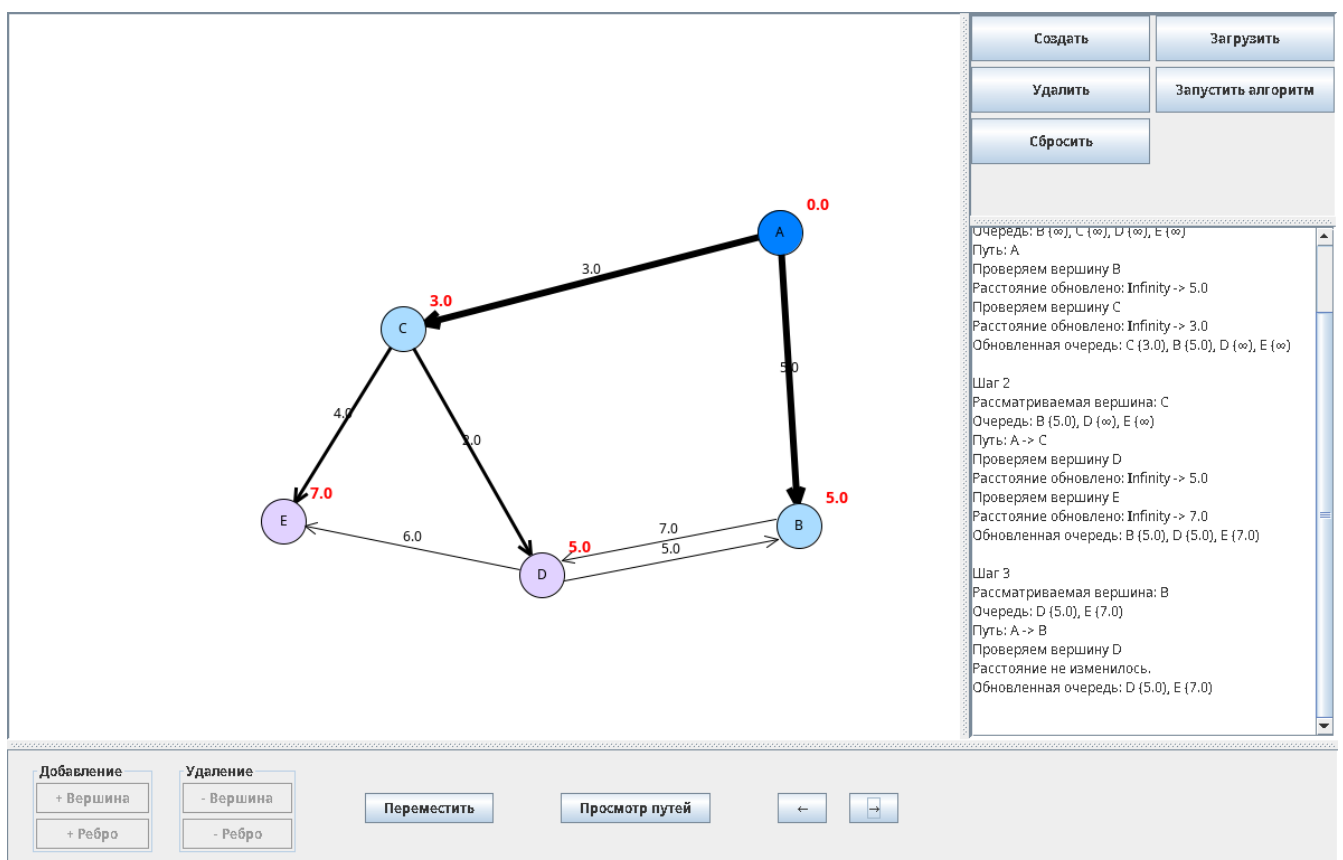


Рисунок 9 – Третий шаг.

Проходим до конца. (см. рис. 10)

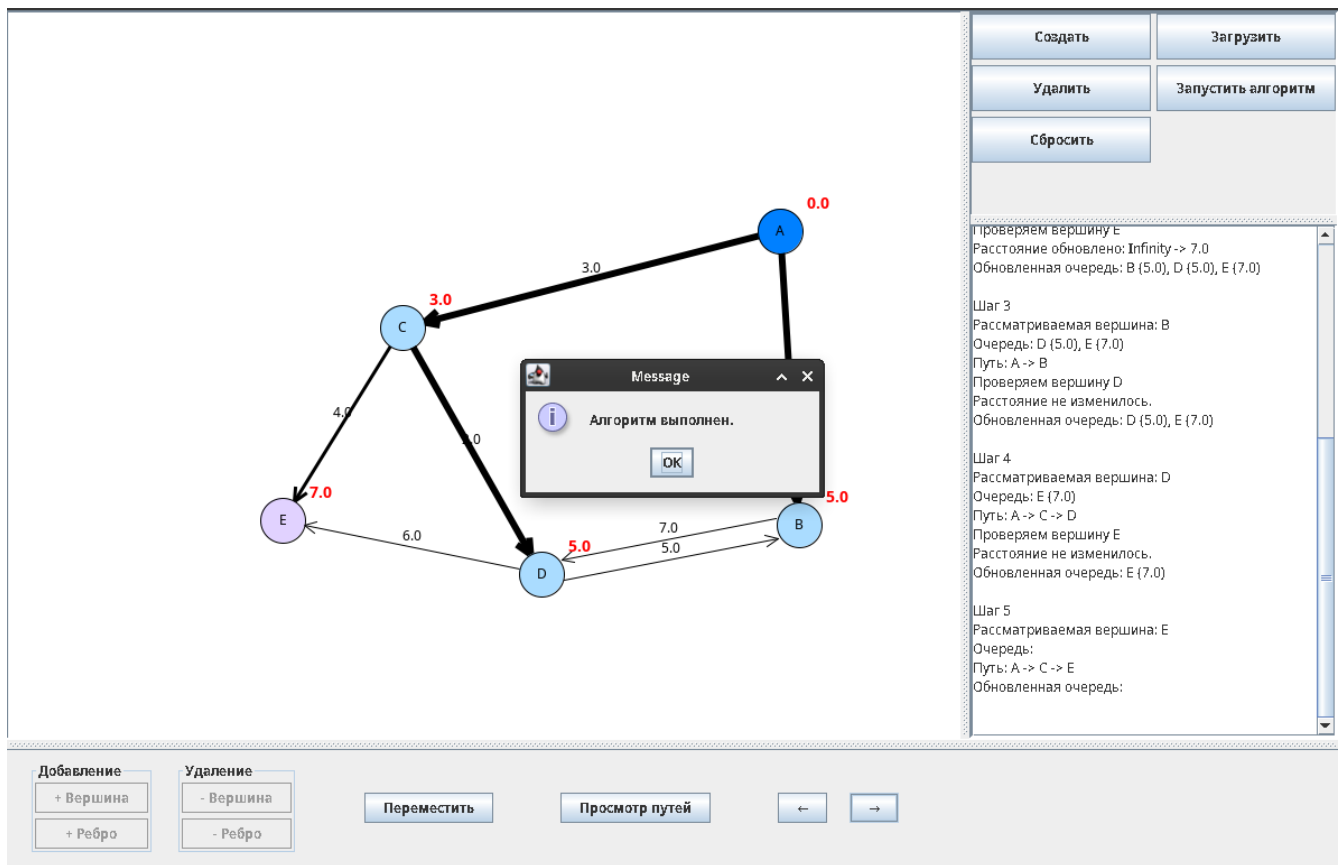


Рисунок 10 – завершение алгоритма.